

Ingeniería de software

Ciclo de vida de desarrollo de software.

Modelos del ciclo de vida del software. (cascada - evolutivo - espiral - de componentes - otros).

Fases de ciclo de vida. Iteraciones e incrementos.

Ciclo de vida del desarrollo del software (SDLC)

- Es un proceso para diseñar y crear software de calidad.
- El objetivo del SDLC es **minimizar los riesgos del proyecto** por medio de una planificación anticipada que permita que el software cumpla las expectativas del cliente durante la fase de producción y postproducción.
- Esta metodología establece una serie de pasos que dividen el proceso de desarrollo de software en tareas que se pueden asignar, completar y medir.

Actividades fundamentales del Ciclo de vida del desarrollo del software

- 1. *Especificación del software.*** Se debe definir la funcionalidad del software y las restricciones en su operación.
- 2. *Diseño e implementación del software.*** Se debe producir software que cumpla su especificación.
- 3. *Validación del software.*** Se debe validar el software para asegurar que hace lo que el cliente desea.
- 4. *Evolución del software.*** El software debe evolucionar para cubrir las necesidades cambiantes del cliente.

Actividades de soporte del Ciclo de vida del desarrollo del software

Ocurren a lo largo del proceso de software y se centran sobre todo en la administración, el seguimiento y el control del proyecto:

Seguimiento y control del proyecto de software, administración del riesgo, aseguramiento de la calidad del software, revisiones técnicas, medición, administración de la configuración del software, administración de la reutilización y preparación y producción del producto del trabajo.

El proceso de ingeniería de software debe ser ágil y adaptable (al problema, al proyecto, al equipo y a la cultura organizacional).

Por tanto, un proceso adoptado para un proyecto puede ser significativamente distinto de otro adoptado para otro proyecto.

Modelos del ciclo de vida del software

- Modelo en cascada
- Desarrollo evolutivo
- Ingeniería del software basada en componentes
- No se excluyen mutuamente y a menudo se utilizan juntos, especialmente para el desarrollo de sistemas grandes. De hecho, el Proceso Unificado, combina elementos de todos estos modelos.

Modelos de proceso prescriptivo (tradicionales)

Definen un conjunto prescrito de elementos del proceso y un flujo predecible para el trabajo del proceso.

Los modelos de proceso prescriptivo fueron propuestos originalmente para poner orden en el caos del desarrollo de software.

La historia indica que estos modelos tradicionales han dado cierta estructura útil al trabajo de ingeniería de software y que constituyen un mapa razonablemente eficaz para los equipos de software.

Modelo en cascada

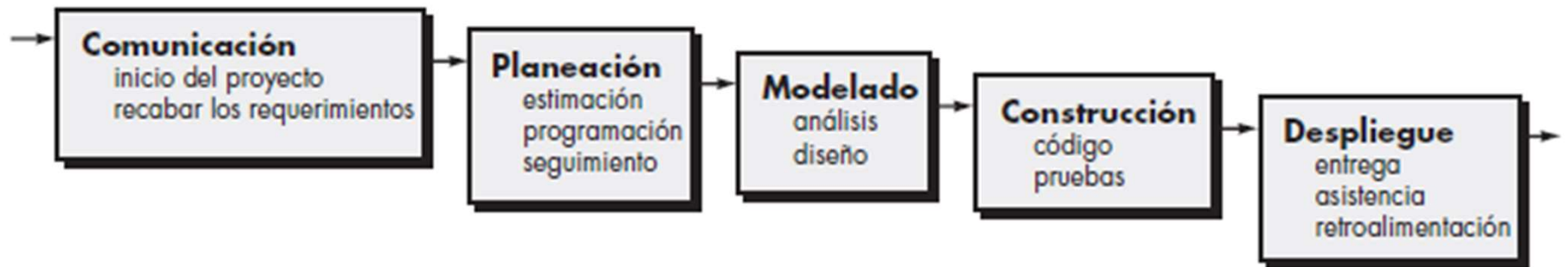
The diagram consists of three horizontal rows. Each row features a colored rounded rectangle on the left containing text, followed by a thin rectangular outline of the same color. The first row is blue, the second is green, and the third is orange. A thin line connects the left side of each colored rectangle to the left side of its corresponding outline, forming a continuous path for each row.

Modelos de proceso incremental

Modelo de proceso evolutivo

Modelo en cascada

- En este modelo del ciclo de vida de un software, se espera a finalizar una etapa para comenzar con la siguiente. Es un flujo de proceso lineal.



Ventajas y desventajas

El modelo de cascada hace que la administración del proyecto sea muy estricta y proporciona un resultado tangible al final de cada fase.

Sin embargo, hay poco margen de cambio una vez que una fase se considera completa, ya que los cambios pueden afectar al tiempo de entrega, al costo y a la calidad del software.

Por lo tanto, el modelo es más adecuado para proyectos de desarrollo de software, donde las tareas se pueden organizar y administrar fácilmente y los requisitos se pueden predefinir con precisión.

Hay veces en las que los requerimientos para cierto problema se comprenden bien:

Cuando el trabajo desde la comunicación hasta el despliegue fluye en forma razonablemente lineal.

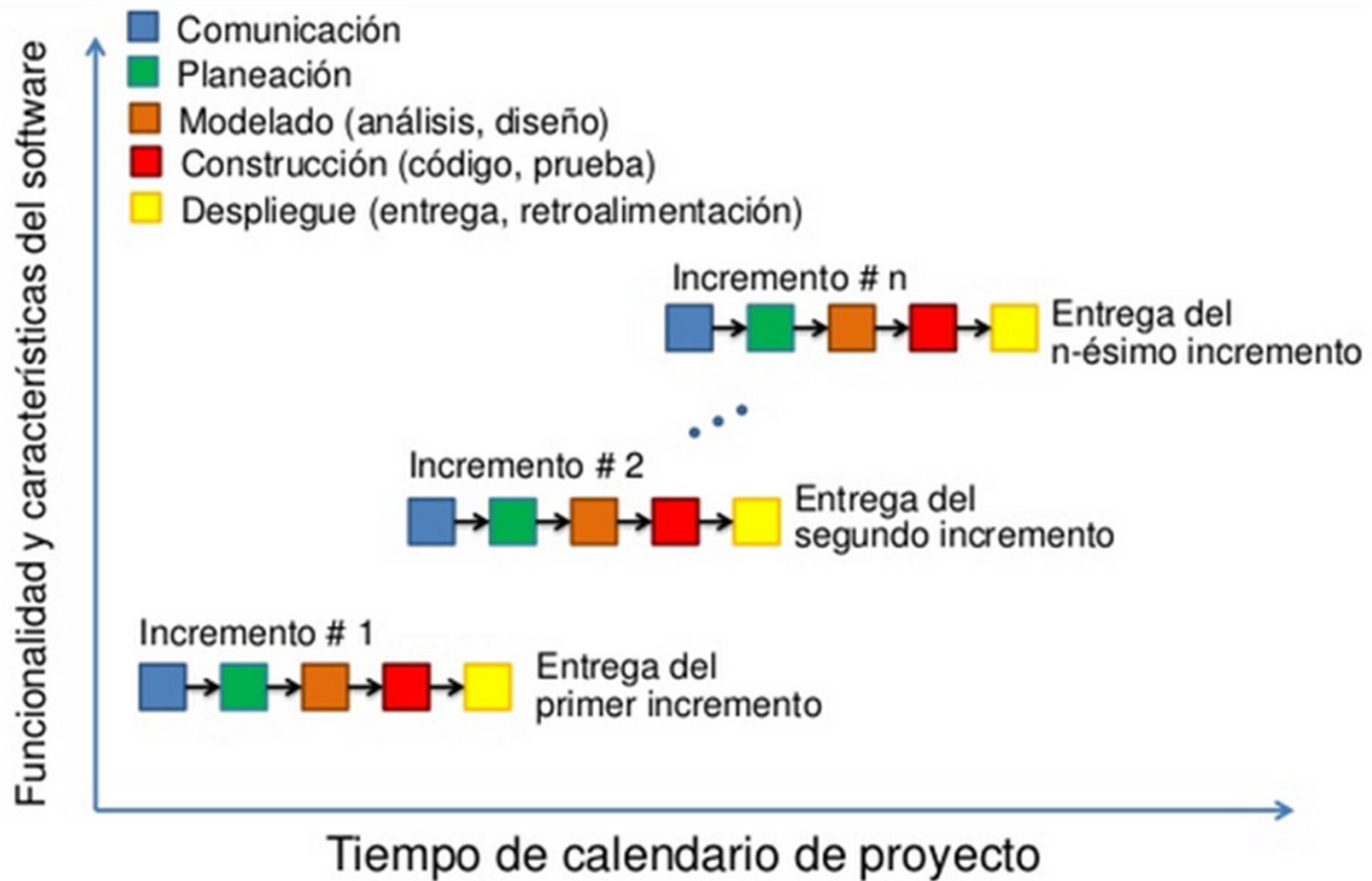
Esta situación se encuentra en ocasiones cuando deben hacerse adaptaciones o mejoras bien definidas a un sistema ya existente (por ejemplo, una adaptación para software de contabilidad que es obligatorio hacer debido a cambios en las regulaciones gubernamentales).

También ocurre en cierto número limitado de nuevos esfuerzos de desarrollo, pero sólo cuando los requerimientos están bien definidos y tienen una estabilidad razonable.

Modelos de proceso incremental

- El modelo incremental combina elementos de los flujos de proceso lineal y paralelo.
- El modelo incremental ejecuta una serie de avances, llamados incrementos, que en forma progresiva dan más funcionalidad al cliente conforme se le entrega cada incremento.

El modelo incremental aplica secuencias lineales en forma escalonada a medida que avanza el calendario de actividades



Características

- Los primeros incrementos son versiones desnudas del producto final, pero proporcionan capacidad que sirve al usuario y también le dan una plataforma de evaluación.
- Cuando se utiliza un modelo incremental, es frecuente que el primer incremento sea el producto fundamental. Es decir, se abordan los requerimientos básicos, pero no se proporcionan muchas características suplementarias (algunas conocidas y otras no). El cliente usa el producto fundamental (o lo somete a una evaluación detallada). Como resultado del uso y/o evaluación, se desarrolla un plan para el incremento que sigue. El plan incluye la modificación del producto fundamental para cumplir mejor las necesidades del cliente, así como la entrega de características adicionales y más funcionalidad. Este proceso se repite después de entregar cada incremento, hasta terminar el producto final.

Ejemplo

- Un software para procesar textos que se elabore con el paradigma incremental quizá entregue en el primer incremento las funciones básicas de administración de archivos, edición y producción del documento; en el segundo dará herramientas más sofisticadas de edición y producción de documentos; en el tercero habrá separación de palabras y revisión de la ortografía; y en el cuarto se proporcionará la capacidad para dar formato avanzado a las páginas.

Debe observarse que el flujo de proceso para cualquier incremento puede incorporar el paradigma del prototipo.

¿Cuándo poder usarlo?

- Cuando los requerimientos iniciales del software están razonablemente bien definidos, pero el alcance general del esfuerzo de desarrollo imposibilita un proceso lineal (cascada). Además, tal vez haya una necesidad imperiosa de dar rápidamente cierta funcionalidad limitada de software a los usuarios y aumentarla en las entregas posteriores de software. En tales casos, se elige un modelo de proceso diseñado para producir el software en incrementos.

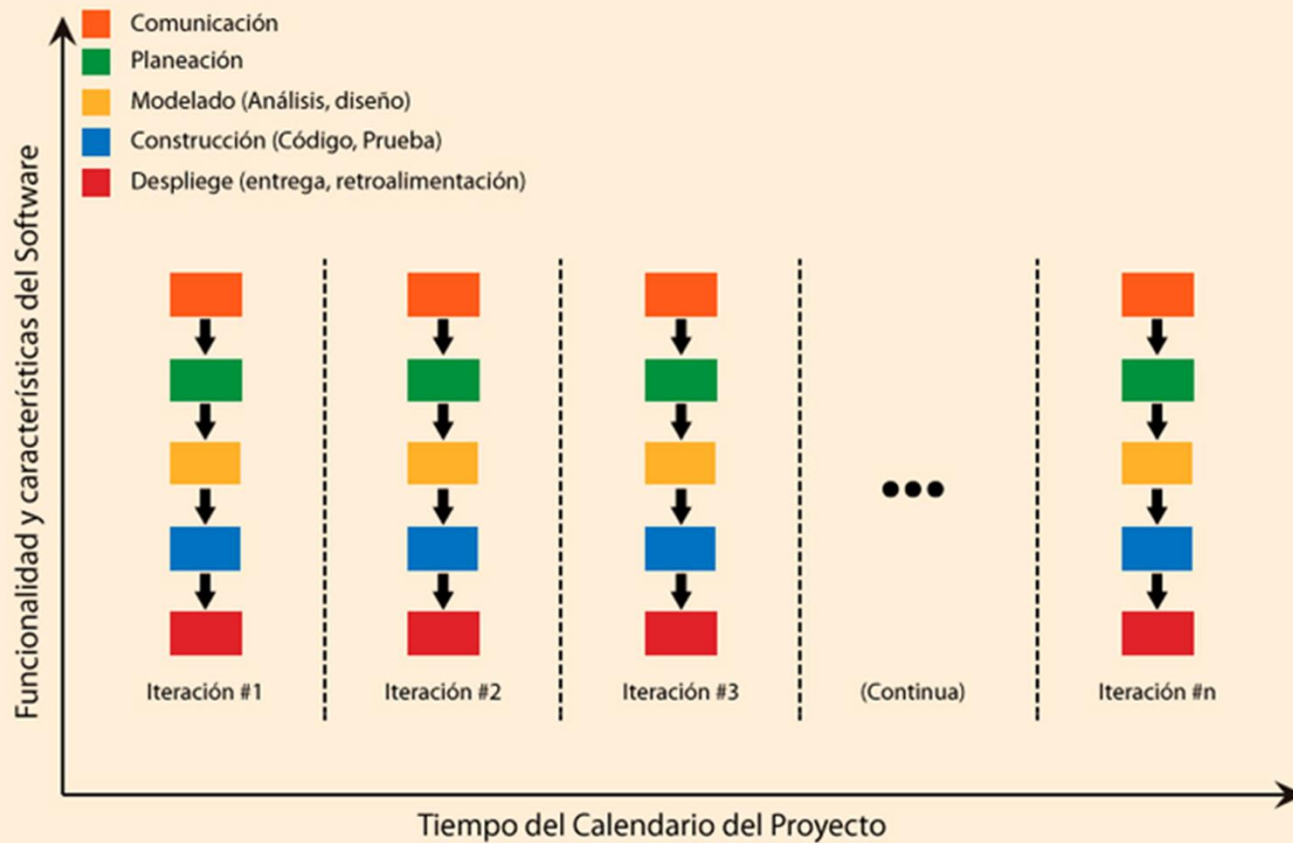
Modelo de proceso evolutivo

Los modelos evolutivos son iterativos

Sugiere que los equipos comienzan el desarrollo de software con un pequeño subconjunto de requisitos. Posteriormente, se mejoran las versiones de manera iterativa a lo largo del tiempo hasta que el software final esté listo para pasar a producción. El equipo produce una nueva versión de software al final de cada iteración.

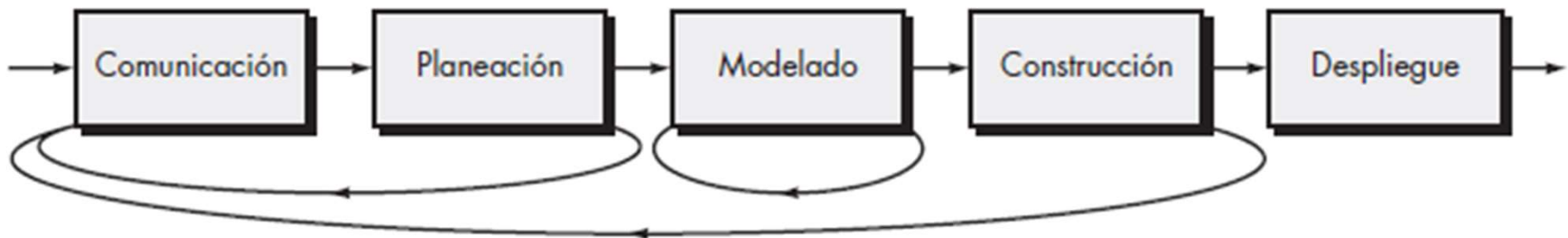
Repite una o más de las actividades antes de pasar a la siguiente

Modelo Iterativo



Ventajas y desventajas

- Es fácil identificar y administrar riesgos, ya que los requisitos pueden cambiar entre cada iteración. Sin embargo, la repetición de los ciclos puede dar lugar a que cambien los objetivos y se subestimen los recursos.



Modelos de proceso especializado

Se aplican cuando se elige un enfoque de ingeniería de software especializado o definido muy específicamente.

Desarrollo basado en componentes

Modelo de métodos formales

Desarrollo basado en componentes

- Construye aplicaciones a partir de fragmentos de software prefabricados.
- Éstos componentes pueden ser módulos, clases orientadas a objetos o paquetes de clases.
- Es de naturaleza evolutiva y demanda un enfoque iterativo para la creación de software.

Modelo de métodos formales

- Los métodos formales permiten especificar, desarrollar y verificar un software por medio del empleo de una notación matemática rigurosa.

Desventajas:

- a) Consume mucho tiempo y es caro.
- b) Se requiere mucha capacitación por parte de los desarrolladores.
- c) Dificulta la comunicación con clientes sin complejidad técnica.

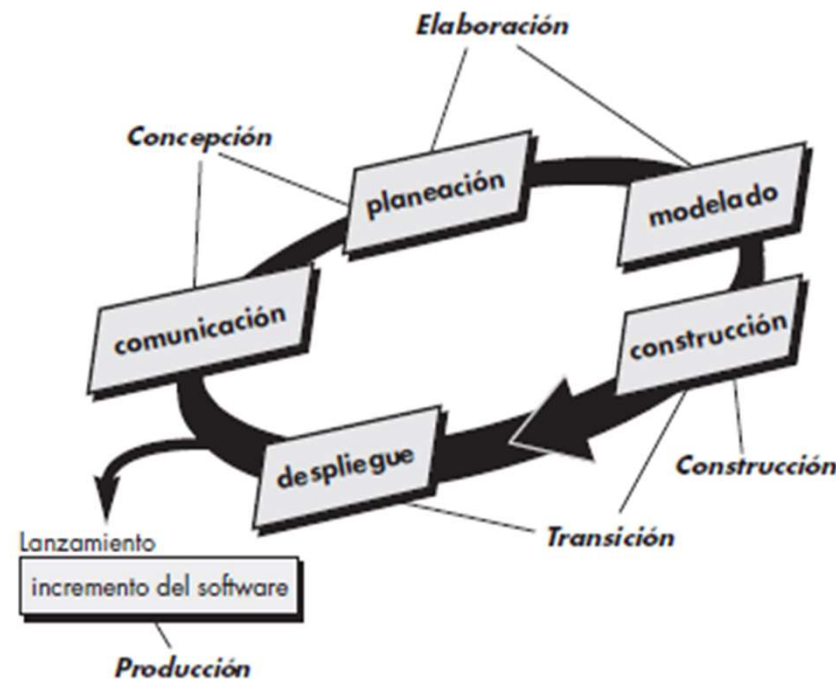
Ventajas:

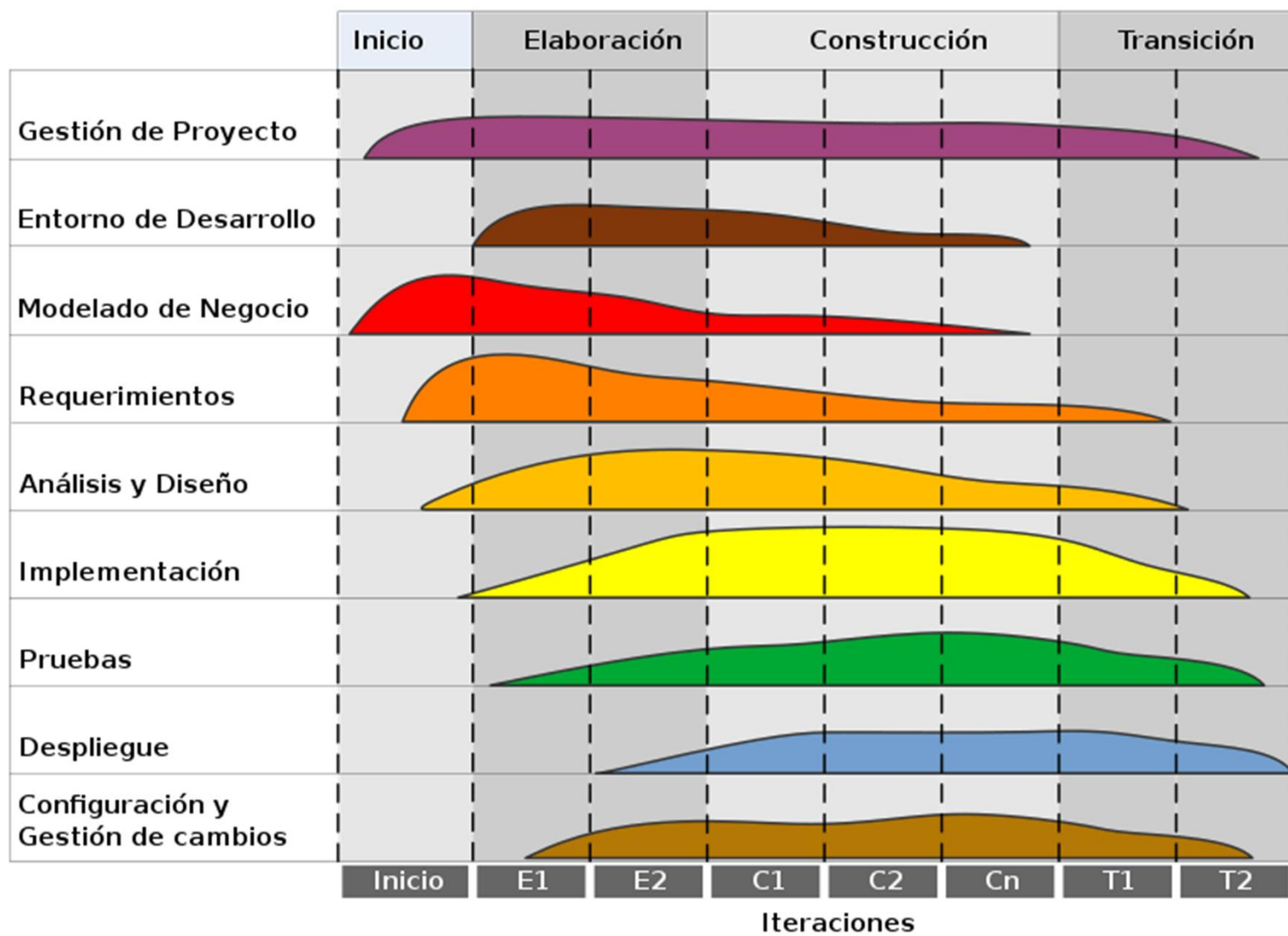
- a) Para sw de alta calidad en seguridad (control electrónico de aeronaves y equipos médicos).
- b) Para evitar grandes pérdidas económicas.

El proceso unificado

Es un proceso impulsado por el caso de uso, centrado en la arquitectura, iterativo e incremental.

- El proceso unificado (PU) y el UML se usan mucho en proyectos de toda clase, orientados a objetos.
- El modelo iterativo e incremental propuesto por el PU puede y debe adaptarse para que satisfaga necesidades específicas del proyecto.





Modelo de desarrollo ágil de software

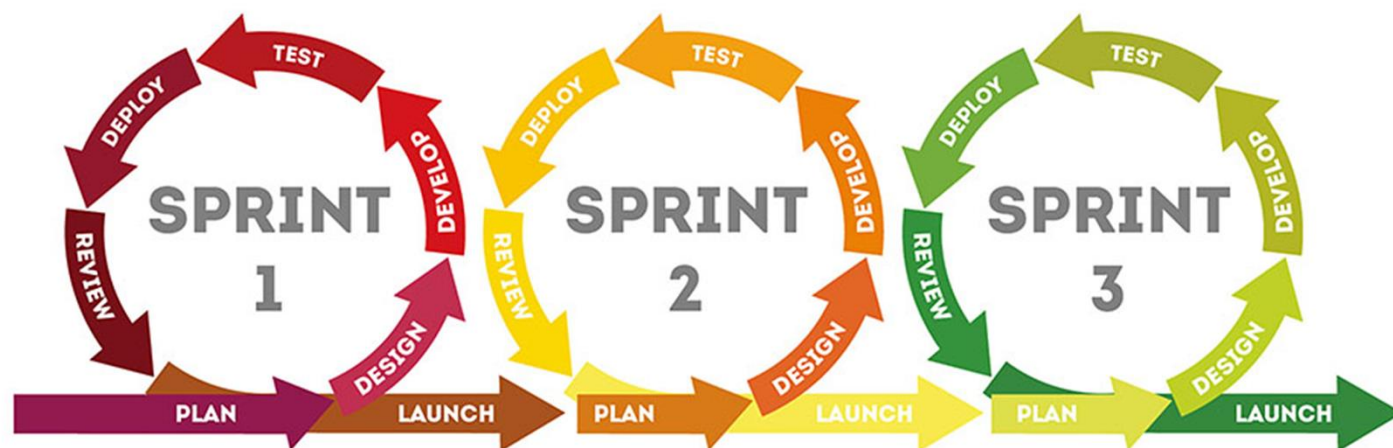
Algunos métodos ágiles de desarrollo son: Scrum, XP, Kanban, OpenUP, entre otros.

Se centra en valorar

- **Individuos e interacciones** sobre procesos y herramientas
- **Software funcionando** sobre documentación extensiva
- **Colaboración con el cliente** sobre negociación contractual
- **Respuesta ante el cambio** sobre seguir un plan

- El modelo ágil organiza las fases del SDLC en varios ciclos de desarrollo.
- El equipo itera a través de las fases rápidamente y solo se hacen pequeños cambios progresivos de software en cada ciclo.
- Los requisitos, planes y resultados se evalúan continuamente para responder con rapidez a los cambios.
- El modelo ágil es iterativo y progresivo, por lo que es más eficiente que otros modelos de procesos.

AGILE



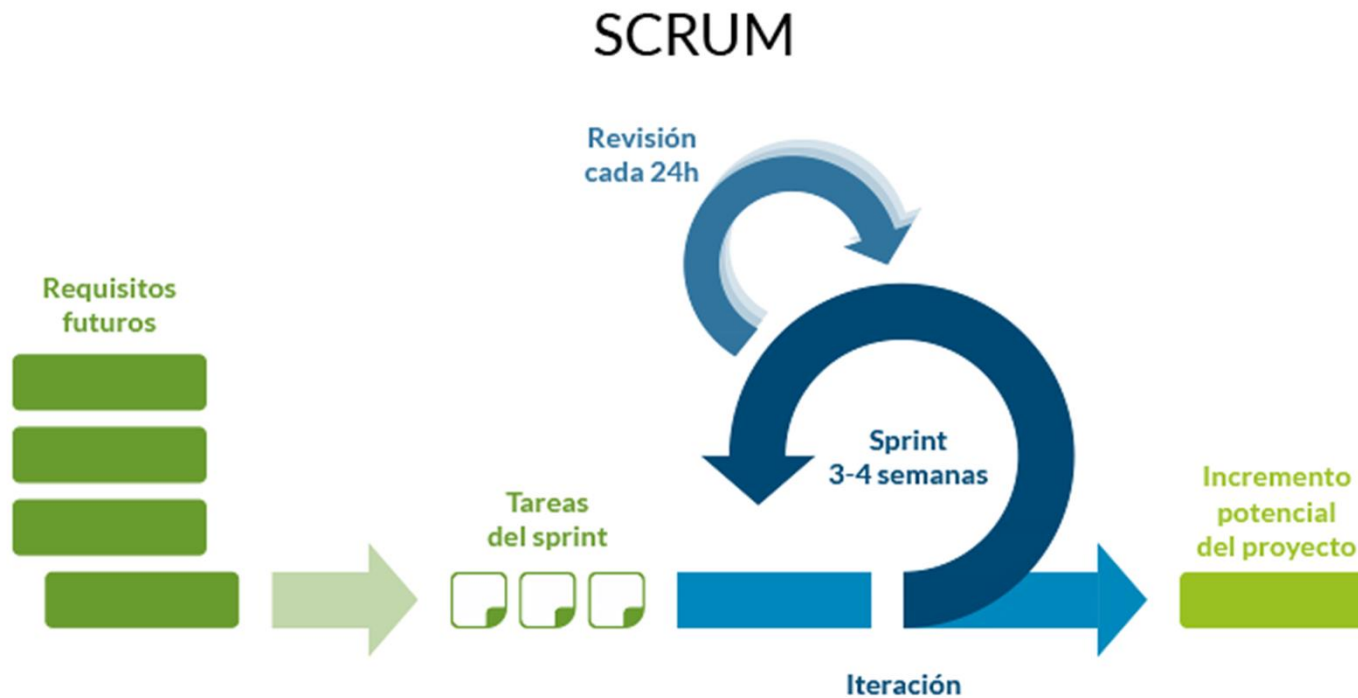
Ventajas y desventajas

Los ciclos rápidos de desarrollo permiten a los equipos identificar y abordar problemas en proyectos complejos desde el principio y antes de que se conviertan en problemas graves.

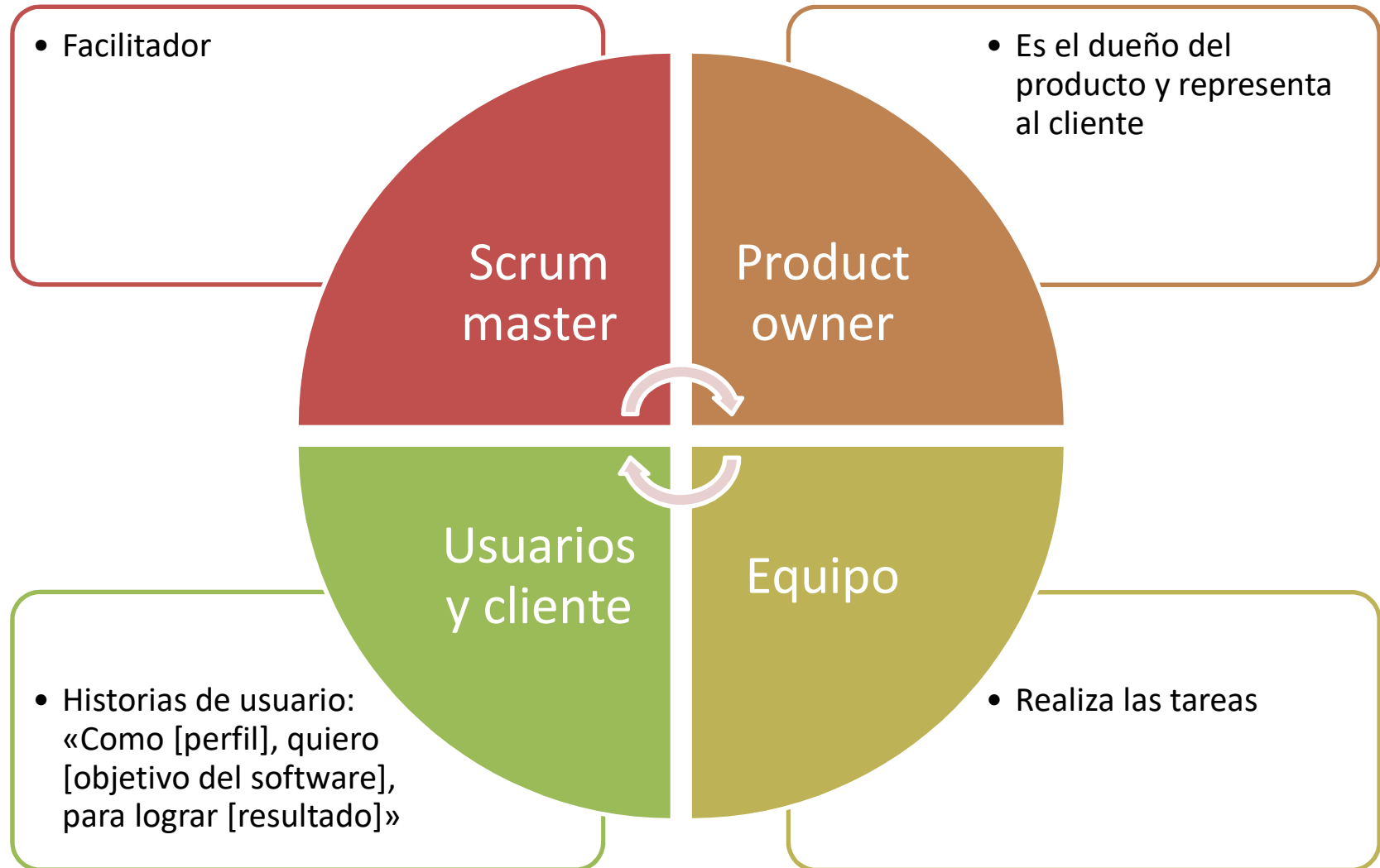
También promueven la participación de los clientes y las partes interesadas para que den su opinión en todo el ciclo de vida del proyecto.

Sin embargo, depender en exceso de la opinión de los clientes puede hacer que los objetivos cambien drásticamente o dejar el proyecto a medias.

Scrum (nace en los '90)



Roles de Scrum



Ejemplo de Historia de usuario

Como encargado de ventas, quiero revisar el desempeño histórico de las ventas, para poder identificar las regiones geográficas y productos de mejor desempeño.

Artefactos de Scrum

Product backlog

Descripciones genéricas de todos los requerimientos y funcionalidades deseables.

Sprint backlog

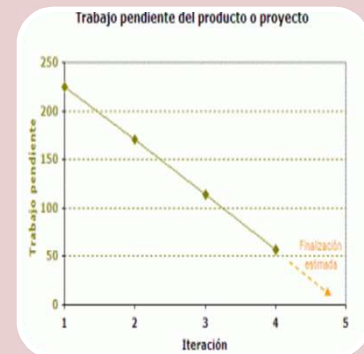
Se establece y compromete el equipo a cumplir los requerimientos funcionales durante el desarrollo del sprint. El sprint dura de 2 a 4 semanas.

Burnup

Progreso del producto

Burndown

Sobre el eje horizontal se muestran los sprints que se desarrollan. Sobre el eje vertical, los requisitos que faltan completar al final de cada sprint.



Reuniones o Ceremonias

Sprint Planning

- Selecciona el trabajo a realizar.
- Interviene el product owner y el equipo que cumplirá los requerimientos.
- Hasta 8 hs.

Daily Scrum

- Reunión diaria con una duración 15 minutos del equipo de desarrollo.

Sprint Review

- Se realiza al final del Sprint. De 1 a 4 hs.
- Se revisa el Incremento, es decir, lo que se realizó durante el Sprint, y se analizan los cambios que tuvo el Product Backlog.
- Product owner, equipo de desarrollo y stakeholders.

Sprint Retrospective

- Al final del Sprint. Scrum Master y equipo de desarrollo. De 1 a 2 hs.
- Los equipos reflexionan sobre qué salió bien y qué podría mejorarse en el próximo sprint.

Kan = “visual” ban=“tarjeta”
(nace en los '50)

LOS 5 PRINCIPIOS DE LA METODOLOGÍA KANBAN



VISUALIZACIÓN

1



PRIORIZACIÓN

2



MEJORA
CONTINUA

3



LIDERAZGO
EN TODOS
LOS NIVELES

4



CALIDAD
GARANTIZADA

5



XP (nace en los '90) centrado en el cliente y orientado a las pruebas



- Comunicación
- Simplicidad
- Retroalimentación
- Valentía/disciplina
- Respeto

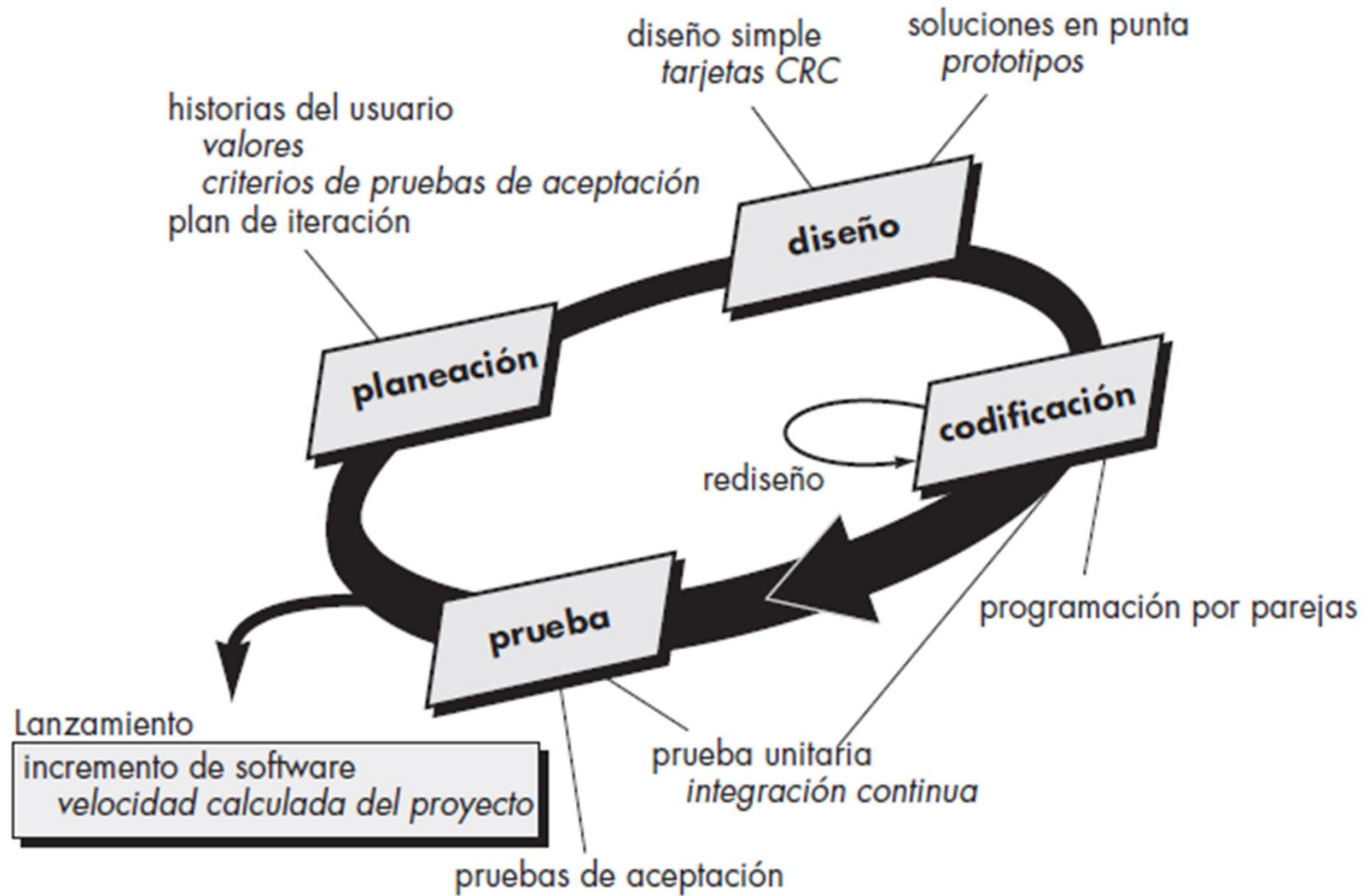
Tarjetas CRC: Una introducción ágil

- Una **Clase** representa una colección de objetos similares, una **Responsabilidad** es algo que una clase conoce o hace, y un **Colaborador** es otra clase con la que interactúa para cumplir con sus responsabilidades.

Student	
Student number Name Address Phone number Enroll in a seminar Drop a seminar Request transcripts	Seminar

¿Cómo se crean los modelos CRC? Realizar los siguientes pasos iterativamente:

- **Buscar clases** . Como regla general, se deben buscar de inmediato entre tres y cinco clases principales, como Estudiante , Seminario y Profesor.
- **Identificar responsabilidades** . Debes preguntarte qué hace una clase y qué información deseas conservar sobre ella. A menudo, identificarás una responsabilidad para que una clase colabore con otra.
- **Definir colaboradores** . Una clase a menudo no cuenta con suficiente información para cumplir con sus responsabilidades. Por lo tanto, debe colaborar (trabajar) con otras clases para realizar el trabajo. Preguntarse: "¿Tiene la clase la capacidad de cumplir con esta responsabilidad?". De no ser así, busque una clase que tenga la capacidad de cumplir con la funcionalidad faltante o la clase que debería cumplirla. Al hacerlo, a menudo descubrirá la necesidad de nuevas responsabilidades en otras clases, e incluso la necesidad de una o dos clases nuevas.
- **Mover las cartas** . Para que todos comprendan mejor el sistema, las cartas deben colocarse sobre la mesa de forma inteligente. Dos cartas que colaboren entre sí deben colocarse juntas, mientras que dos cartas que no colaboren deben colocarse separadas. Además, cuanto más colaboren dos cartas, más cerca deben estar sobre la mesa. Al tener las cartas que colaboran juntas, es más fácil comprender las relaciones entre las clases.



CRC (Clase-Responsabilidad-Colaboración)